

③ 如何捕捉空间相关性

③ 如何捕捉空间相关性

课程回顾

- 时空人工智能：时空属性建模、多源数据融合、支持多种任务

多源数据融合、支持多种任务
邻近性、层次性、距离)、时间属性 (临近性、)

- 时空人工智能：时空属性建模、多源数据融合、支持多种任务
- 时空数据的特性：空间属性（邻近性、层次性、距离）、时间属性（临近性、周期性、趋势性）

多源数据融合、支持多种任务
近性、层次性、距离)、时间属性 (临近性、

```
graph LR; A[数据集] --> B[时空模型构建]; B --> C[时空 AI 模型]; C --> D[模型部署]; D --> E[应用];
```

- ```

graph LR
 A[原始时空数据] --> B[数据转换]
 B --> C[标准时空数据集]
 C --> D[时空模型构建]
 D --> E[时空 AI 模型]
 E --> F[模型部署]
 F --> G[应用]

```

多源数据融合、支持多种任务  
(空间属性(临近性、层次性、距离)、时间属性(临近性、周期性))

```
graph LR; A[数据集] --> B[时空模型构建]; B --> C[时空 AI 模型]; C --> D[模型部署]; D --> E[应用];
```

数据集 → 时空模型构建 → 时空 AI 模型 → 模型部署 → 应用

- ```

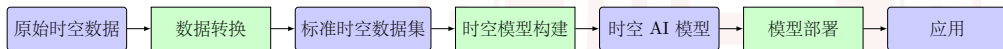
graph LR
    A[原始时空数据] --> B[数据转换]
    B --> C[标准时空数据集]
    C --> D[时空模型构建]
    D --> E[时空 AI 模型]
    E --> F[模型部署]
    F --> G[应用]

```


课程回顾

- 时空人工智能：时空属性建模、多源数据融合、支持多种任务
- 时空数据的特性：空间属性（邻近性、层次性、距离）、时间属性（临近性、周期性、趋势性）

- 时空 AI 框架



- 时空特征工程 + 小模型
- 时空深度学习

③ 如何捕捉空间相关性

学习目标

本讲目标

- ## ① 掌握时间相关性的捕捉方法：从传统时序模型到现代深度学习

学习目标

本讲目标

- 1 掌握时间相关性的捕捉方法：从传统时序模型到现代深度学习
- 2 理解空间相关性的建模技术：图神经网络的核心思想

本讲目标

- 1 掌握时间相关性的捕捉方法：从传统时序模型到现代深度学习
- 2 理解空间相关性的建模技术：图神经网络的核心思想
- 3 熟悉时空融合的先进模型架构

③ 如何捕捉空间相关性

② 如何捕捉时间相关性

MLP 预测模型

多层感知机 (MLP)

- 将时间序列视为特征向量
- 通过全连接层学习非线性映射
- 优点：简单易实现，训练快速
- 局限：忽略序列顺序，难以捕捉长期依赖

核心思想

$$y_{t+1} = f(x_t, x_{t-1}, \dots, x_{t-w+1})$$

- w : 滑动窗口大小
- f : MLP 学习的非线性函数

Diagram illustrating a sequence of variables $x_3, x_4, x_5, \dots$ in blue boxes, representing a sequence of states or data points.

Diagram illustrating a sequence of variables $x_3, x_4, x_5, \dots$. The variable x_3 is highlighted with a red border.

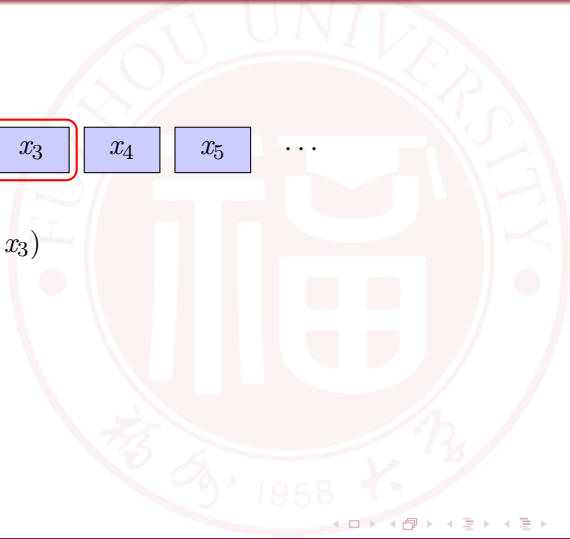


Diagram illustrating a sequence of variables $x_3, x_4, x_5, \dots$. The variables x_3 and x_4 are highlighted with a red box, indicating they are the current focus of the analysis.

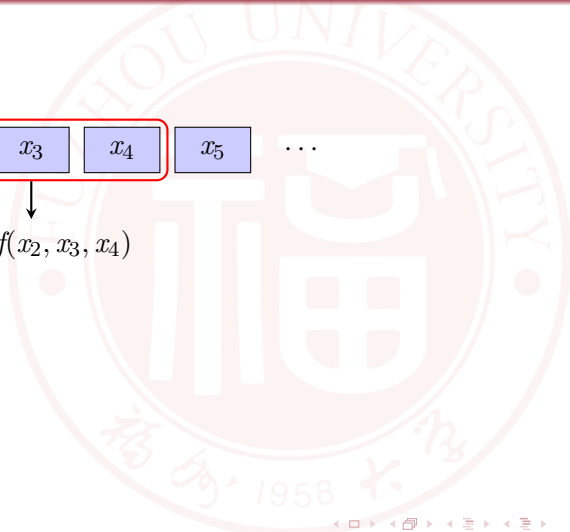
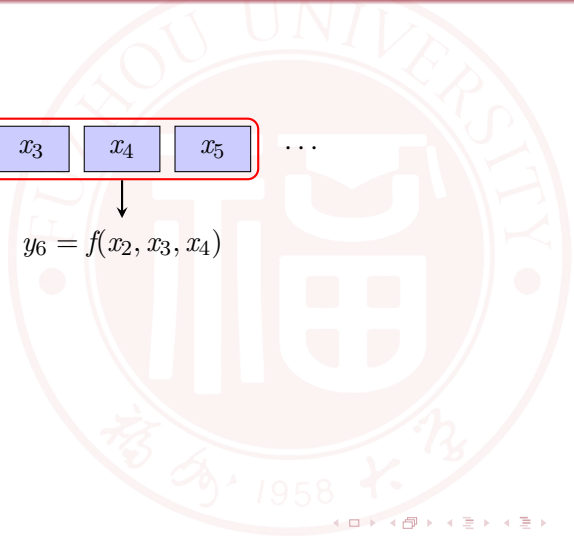
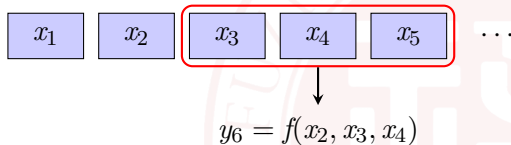


Diagram illustrating a sequence of variables $x_3, x_4, x_5, \dots$ arranged horizontally. The variables x_3, x_4, x_5 are enclosed in boxes, and the entire sequence is highlighted by a red rectangular border.

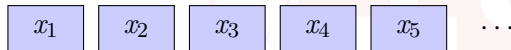




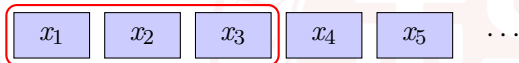
应用场景

适用于短期预测，序列长度固定的简单任务。

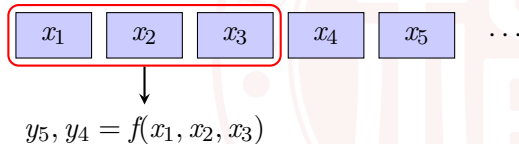
方法一



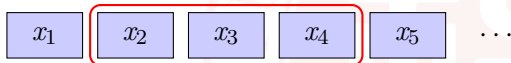
方法一



方法一

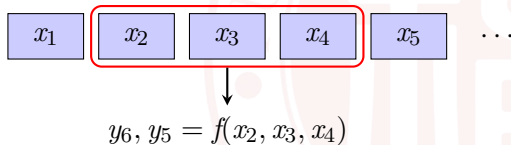


方法一

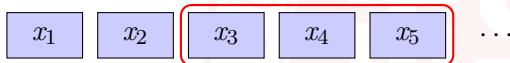


滑动窗口之多步预测

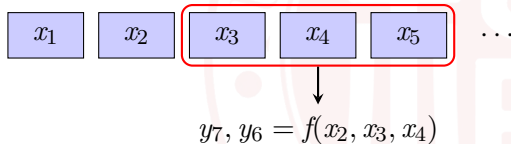
方法一



方法一



方法一



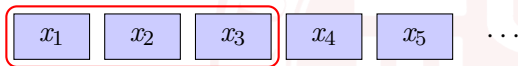
方法二

递归预测：将前一步预测结果作为下一步输入，适用于长期预测，但误差易累积。



方法二

递归预测：将前一步预测结果作为下一步输入，适用于长期预测，但误差易累积。



方法二

A diagram showing a sequence of variables $x_1, x_2, x_3, x_4, x_5, \dots$. The variables x_1, x_2, x_3 are grouped together by a red rounded rectangle. An arrow points downwards from the bottom center of this red box.

$$y_4 = f(x_1, x_2, x_3)$$

方法二

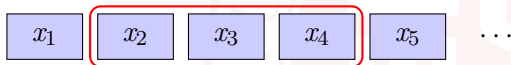
A diagram showing a sequence of variables $x_1, x_2, x_3, x_4, x_5, \dots$ in light blue boxes. A red rounded rectangle highlights the first three elements, x_1, x_2, x_3 .

$$y_4 = f(x_1, x_2, x_3)$$

$$y_5 = f(x_2, x_3, y_4)$$

方法二

递归预测：将前一步预测结果作为下一步输入，适用于长期预测，但误差易累积。



方法二

A diagram illustrating a sequence of inputs $x_1, x_2, x_3, x_4, x_5, \dots$. The inputs x_2, x_3, x_4 are grouped together in a red box, and an arrow points from this group to the function $y_5 = f(x_2, x_3, x_4)$.

方法二

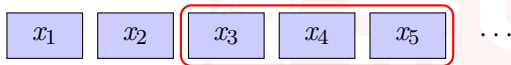
A sequence of variables $x_1, x_2, x_3, x_4, x_5, \dots$ is shown in light blue boxes. A red box highlights the subsequence x_2, x_3, x_4 .

$$y_5 = f(x_2, x_3, x_4)$$

$$y_6 = f(x_3, x_4, y_5)$$

方法二

递归预测：将前一步预测结果作为下一步输入，适用于长期预测，但误差易累积。



方法二

A sequence of variables $x_1, x_2, x_3, x_4, x_5, \dots$ is shown. The variables x_3, x_4, x_5 are enclosed in a red box, indicating they are the focus of the current step.

$$y_6 = f(x_3, x_4, x_5)$$

方法二

A sequence of variables $x_1, x_2, x_3, x_4, x_5, \dots$ is shown. The variables x_3, x_4, x_5 are enclosed in a red box, indicating they are the focus of the current step in the algorithm.

$$y_6 = f(x_3, x_4, x_5)$$

$$y_7 = f(x_4, x_5, y_6)$$

忽略预测间依赖，增加预测步数需要重新训练
误差易累积，串行计算开销大。

- 方法一：并行预测，速度快，但忽略预测间依赖，增加预测步数需要重新训练模型。
- 方法二：捕捉预测间依赖，但误差易累积，串行计算开销大。

RNN

RNN 基本结构

- 引入隐状态 h_t 作为记忆单元
- 当前输出依赖当前输入和前一时刻状态
- 更新公式:

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

$$y_t = g(W_y h_t + c)$$

梯度问题

- **梯度消失**: 难以学习长期依赖
- **梯度爆炸**: 训练不稳定

LSTM 核心机制

- 输入门：控制新信息的流入
- 遗忘门：控制旧信息的遗忘
- 输出门：控制当前输出的信息
- 细胞状态：长期记忆的传输通道

门控更新公式

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

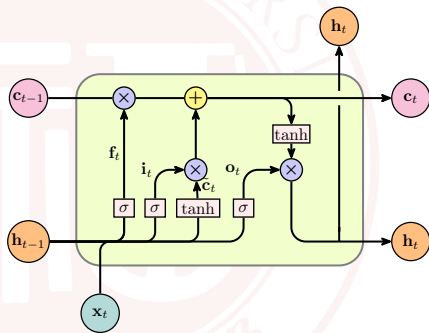
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$



LSTM 单元结构示意图

LSTM 的简化版本，计算效率更高

GRU vs LSTM

- **参数更少**: GRU 只有两个门 (LSTM 三个门)
- **计算更快**: 结构更简单, 训练速度更快
- **性能相近**: 在很多任务上表现与 LSTM 相当

GRU 门控机制

- 更新门：平衡新旧信息
- 重置门：控制历史信息利用程度

更新公式

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t])$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

GRU

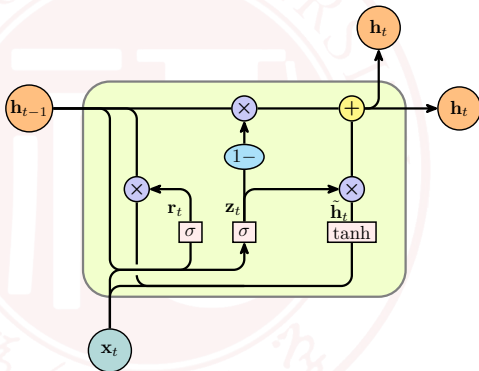
LSTM 的简化版本，计算效率更高

GRU vs LSTM

- 参数更少：GRU 只有两个门（LSTM 三个门）
- 计算更快：结构更简单，训练速度更快
- 性能相近：在很多任务上表现与 LSTM 相当

GRU 门控机制

- 更新门：平衡新旧信息
- 重置门：控制历史信息的利用程度



GRU 单元结构示意图

一维时序卷积

- 卷积核在时间轴上滑动
- 提取局部时间模式特征
- 优点：并行计算，训练高效
- 局限：感受野有限，难以建模长期依赖

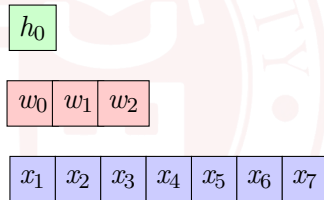


图 1: 一维时序卷积示意图

一维时序卷积

- 卷积核在时间轴上滑动
- 提取局部时间模式特征
- 优点：并行计算，训练高效
- 局限：感受野有限，难以建模长期依赖

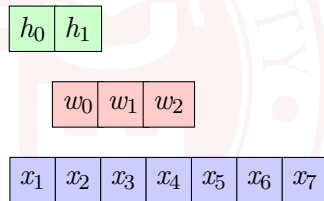


图 1: 一维时序卷积示意图

CNN 与时序卷积

一维时序卷积

- 卷积核在时间轴上滑动
- 提取局部时间模式特征
- 优点：并行计算，训练高效
- 局限：感受野有限，难以建模长期依赖

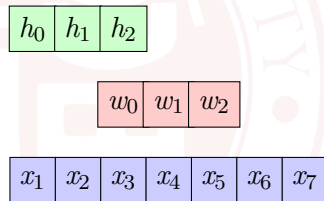


图 1: 一维时序卷积示意图

CNN 与时序卷积

一维时序卷积

- 卷积核在时间轴上滑动
- 提取局部时间模式特征
- 优点：并行计算，训练高效
- 局限：感受野有限，难以建模长期依赖

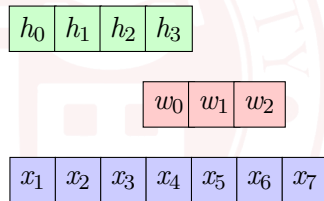


图 1: 一维时序卷积示意图

一维时序卷积

- 卷积核在时间轴上滑动
- 提取局部时间模式特征
- 优点：并行计算，训练高效
- 局限：感受野有限，难以建模长期依赖

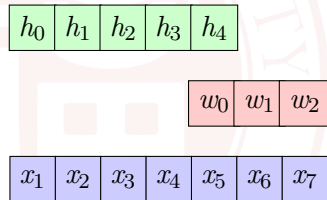
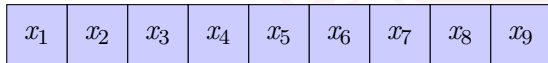


图 1: 一维时序卷积示意图

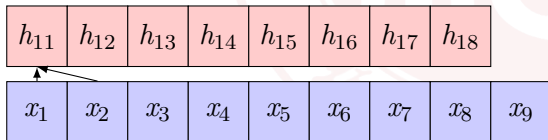
空洞卷积 (Dilated CNN)

- 不增加参数的前提下，扩大感受野
- 适合捕捉多尺度时序模式



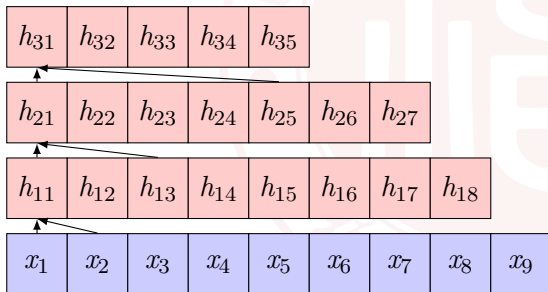
空洞卷积 (Dilated CNN)

- 不增加参数的前提下，扩大感受野
- 适合捕捉多尺度时序模式



空洞卷积 (Dilated CNN)

- 不增加参数的前提下，扩大感受野
- 适合捕捉多尺度时序模式



基础时序模型特性对比

模型	参数量	主要优点	主要局限
MLP	少	简单快速，易于实现	忽略序列顺序，难建模长期依赖
CNN	中等	并行计算，训练高效	感受野有限，需多层堆叠
RNN	少	天然序列结构	梯度消失/爆炸问题
LSTM	多	解决长期依赖，记忆能力强	参数多，训练慢
GRU	中等	平衡性能与效率	极长序列仍有挑战

应用场景分析

模型选择指南

- 简单任务: MLP/CNN (计算效率优先)
- 中等长度序列: GRU (平衡性能与效率)
- 长序列复杂模式: LSTM (精度优先)
- 实时应用: CNN/GRU (推理速度要求高)

② 如何捕捉时间相关性

基础时序模型

现代时序模型

TCN 与时序卷积网络

标准 Transformer 模型

前沿高阶模型

本节小结

③ 如何捕捉空间相关性

为什么门控机制重要?

- **动态调节**：根据输入自适应控制信息流（如抑制噪声）
- **解决梯度问题**：避免深层网络梯度消失（比纯残差连接更有效）
- **增强表达力**：捕获更复杂的时序依赖（如突变点、周期性变化）

门控 TCN 的实现与优势

门控 TCN 架构

- 残差连接 + 门控卷积: $h_t^{(l)} = \text{ReLU}(g \odot F(h_t^{(l-1)})) + h_t^{(l-1)}$
- 门控层位置: 在卷积层后、激活函数前
- 与标准 TCN 对比: 标准 TCN 仅用 $h_t^{(l)} = \text{ReLU}(F(h_t^{(l-1)})) + h_t^{(l-1)}$

关键性能优势

- 长序列稳定性: 序列长度 512 时, 门控 TCN MAE 仅上升 3.2% (标准 TCN 上升 12.7%)
- 突变点处理: 对异常值 (如停电事件) 响应更快 (延迟降低 27%)
- 计算效率: 门控层额外开销 <5% (参数量仅增加 0.1M)

主流变体

- **Autoformer**: 序列分解 (趋势 + 周期 + 残差)
 - 趋势: $\mathcal{F}^{-1}(\hat{X} \odot \mathbb{K}_{\text{low freq}})$

主流变体

- **Autoformer**: 序列分解 (趋势 + 周期 + 残差)
 - 趋势: $\mathcal{F}^{-1}(\hat{X} \odot \mathbb{K}_{\text{low freq}})$
- **PatchTST**: 序列切片为 patch ($L = n/p$)
 - 无需额外位置编码

主流变体

- **Autoformer**: 序列分解 (趋势 + 周期 + 残差)
 - 趋势: $\mathcal{F}^{-1}(\hat{X} \odot \mathbb{K}_{\text{low freq}})$
- **PatchTST**: 序列切片为 patch ($L = n/p$)
 - 无需额外位置编码
- **Non-stationary**: $X_{\text{non-st}} = X + t$

主流变体

- **Autoformer**: 序列分解 (趋势 + 周期 + 残差)
 - 趋势: $\mathcal{F}^{-1}(\hat{X} \odot \mathbb{K}_{\text{low freq}})$
- **PatchTST**: 序列切片为 patch ($L = n/p$)
 - 无需额外位置编码
- **Non-stationary**: $X_{\text{non-st}} = X + t$

适用场景

主流变体

- **Autoformer**: 序列分解 (趋势 + 周期 + 残差)
 - 趋势: $\mathcal{F}^{-1}(\hat{X} \odot \mathbb{K}_{\text{low freq}})$
- **PatchTST**: 序列切片为 patch ($L = n/p$)
 - 无需额外位置编码
- **Non-stationary**: $X_{\text{non-st}} = X + t$

适用场景

- 短期预测: 传统 Transformer ($n < 100$)
- 中期预测: Autoformer/PatchTST ($100 \leq n \leq 500$)

- 短期预测: 传统 Transformer ($n < 100$)
- 中期预测: Autoformer/PatchTST ($100 \leq n \leq 500$)
- 长序列: Informer/FEDformer ($n > 500$)

② 如何捕捉时间相关性

基础时序模型

现代时序模型

前沿高阶模型

高效架构创新: Informer, Pyraformer

频率域创新: FEDformer

多元关联创新: Crossformer

本节小结

③ 如何捕捉空间相关性

问题：长序列预测的计算瓶颈

核心挑战

- 传统 **Transformer**：自注意力计算复杂度 $O(L^2)$
- 实际限制： $L > 500$ 时显存溢出（如 ETTh1 数据集）
- 目标：将复杂度降至 $O(L \log L)$

Informer 三大创新机制

问题：长序列预测的计算瓶颈

核心挑战

- 传统 **Transformer**：自注意力计算复杂度 $O(L^2)$
- 实际限制： $L > 500$ 时显存溢出（如 ETTh1 数据集）
- 目标：将复杂度降至 $O(L \log L)$

Informer 三大创新机制

- ① **ProbSparse 自注意力**：概率稀疏化

问题：长序列预测的计算瓶颈

核心挑战

- 传统 **Transformer**：自注意力计算复杂度 $O(L^2)$
- 实际限制： $L > 500$ 时显存溢出（如 ETTh1 数据集）
- 目标：将复杂度降至 $O(L \log L)$

Informer 三大创新机制

- ① **ProbSparse 自注意力**：概率稀疏化
- ② **自注意力蒸馏**：序列长度压缩

问题：长序列预测的计算瓶颈

核心挑战

- 传统 **Transformer**：自注意力计算复杂度 $O(L^2)$
- 实际限制： $L > 500$ 时显存溢出（如 ETTh1 数据集）
- 目标：将复杂度降至 $O(L \log L)$

Informer 三大创新机制

- ① **ProbSparse 自注意力**：概率稀疏化
- ② **自注意力蒸馏**：序列长度压缩
- ③ **生成式解码器**：一次性预测未来

核心机制

- 关键参数: $u = c \cdot \ln L$ ($c = 5$, L 为序列长度)

核心机制

- 关键参数: $u = c \cdot \ln L$ ($c = 5$, L 为序列长度)
- 操作步骤

核心机制

- 关键参数: $u = c \cdot \ln L$ ($c = 5$, L 为序列长度)
- 操作步骤
 - ① 采样 u 个 Key 来计算注意力分数 $S = \text{softmax}(QK^T / \sqrt{d_k})$

核心机制

- 关键参数: $u = c \cdot \ln L$ ($c = 5$, L 为序列长度)
- 操作步骤
 - ① 采样 u 个 Key 来计算注意力分数 $S = \text{softmax}(QK^T/\sqrt{d_k})$
 - ② 选择 u 个最高概率的 查询 (而非键!)

ProbSparse 自注意力：概率稀疏化原理

核心机制

- 关键参数： $u = c \cdot \ln L$ （ $c = 5$, L 为序列长度）
- 操作步骤
 - ① 采样 u 个 Key 来计算注意力分数 $S = \text{softmax}(QK^T/\sqrt{d_k})$
 - ② 选择 u 个最高概率的 查询（而非键!）
 - ③ 仅计算 u 个查询与所有键的注意力

核心机制

- **关键参数：** $u = c \cdot \ln L$ ($c = 5$, L 为序列长度)
- **操作步骤**
 - ① 采样 u 个 Key 来计算注意力分数 $S = \text{softmax}(QK^T/\sqrt{d_k})$
 - ② 选择 u 个最高概率的 查询（而非键!）
 - ③ 仅计算 u 个查询与所有键的注意力
- **复杂度：** $O(L \log L)$ vs $O(L^2)$

ProbSparse 自注意力：概率稀疏化原理

核心机制

- 关键参数： $u = c \cdot \ln L$ （ $c = 5$ ， L 为序列长度）
- 操作步骤
 - ① 采样 u 个 Key 来计算注意力分数 $S = \text{softmax}(QK^T/\sqrt{d_k})$
 - ② 选择 u 个最高概率的 查询（而非键!）
 - ③ 仅计算 u 个查询与所有键的注意力
- 复杂度： $O(L \log L)$ vs $O(L^2)$

信息保留率

- $S_{\text{full}} \sim \mathcal{N}(0, d_k)$, $\mathbb{E} [\sum_{i=1}^u S_{\text{top-}u}(i)] \geq 1 - \frac{1}{L}$
- 示例： $L = 1000$ 时， $u = 5 \cdot \ln(1000) \approx 34.5$ ， 仅需计算 35 个查询

自注意力蒸馏机制：序列长度压缩

蒸馏操作原理

- 每个注意力层后添加卷积池化： $X_{j+1} = \text{MaxPool}(\text{ELU}(\text{Conv1d}(X_j)))$
- 关键效果：序列长度每层减半
- 复杂度影响：

原始复杂度 $\rightarrow O(L^2)$

蒸馏后 $\rightarrow O((L/2)^2)$ (下一层输入)

3 层蒸馏后 $\rightarrow O((L/8)^2)$

协同效应

- ProbSparse 处理 L 长度序列： $O(L \log L)$
- 蒸馏后序列长度变为 $L/8$ ，ProbSparse 计算量降至 $O((L/8) \log(L/8))$

工作流程

- ① 输入：长序列 $X \in \mathbb{R}^{L \times d}$



工作流程

- 1 输入: 长序列 $X \in \mathbb{R}^{L \times d}$
- 2 蒸馏层: $X \rightarrow X_1 \rightarrow X_2 \rightarrow X_3$ ($L \rightarrow L/8$)

工作流程

- 1 输入: 长序列 $X \in \mathbb{R}^{L \times d}$
- 2 蒸馏层: $X \rightarrow X_1 \rightarrow X_2 \rightarrow X_3$ ($L \rightarrow L/8$)
- 3 ProbSparse 注意力: 在 X_3 上计算 $O(L \log L)$ 注意力

工作流程

- 1 输入: 长序列 $X \in \mathbb{R}^{L \times d}$
- 2 蒸馏层: $X \rightarrow X_1 \rightarrow X_2 \rightarrow X_3$ ($L \rightarrow L/8$)
- 3 **ProbSparse** 注意力: 在 X_3 上计算 $O(L \log L)$ 注意力
- 4 生成式解码: 一次性输出 Y ($O(1)$)

完整机制：三大创新的协同效应

工作流程

- ① 输入：长序列 $X \in \mathbb{R}^{L \times d}$
- ② 蒸馏层： $X \rightarrow X_1 \rightarrow X_2 \rightarrow X_3$ ($L \rightarrow L/8$)
- ③ **ProbSparse** 注意力：在 X_3 上计算 $O(L \log L)$ 注意力
- ④ 生成式解码：一次性输出 Y ($O(1)$)

复杂度综合分析

$$\begin{aligned} \text{总复杂度} &= \underbrace{O(L \log L)}_{\text{ProbSparse}} + \underbrace{O(L \log L)}_{\text{蒸馏}} + \underbrace{O(1)}_{\text{解码}} \\ &= O(L \log L) \end{aligned}$$

ETTh1 数据集性能实测 (序列长度 = 1000)

关键指标对比

模型	MAE	推理时间	内存	序列长度
Transformer	OOM	-	溢出	$L = 100$
Informer	0.678	0.12s	1.8GB	$L = 1000$
Informer+ 蒸馏	0.651	0.08s	1.2GB	$L = 10000$

核心优势

- 长序列处理: $L > 500$ 时唯一可行方案 (Transformer 直接 OOM)
- 计算效率: 比 Transformer 快 $15.3\times$ ($L = 1000$)
- 精度: 比 TCN MAE 低 12.1%
- 扩展性: 蒸馏后可处理 $L = 10000$ (传统模型无法实现)

核心挑战

- 传统 Transformer: 自注意力计算复杂度 $O(L^2)$
- 实际限制: $L > 500$ 时显存溢出 (如 ETTm1 数据集)
- 关键瓶颈: 信号传播路径 $O(L)$ (需 L 步传播)

Pyraformer 的突破性目标

- 时间复杂度：降至 $O(L)$ （线性）

核心挑战

- 传统 Transformer: 自注意力计算复杂度 $O(L^2)$
- 实际限制: $L > 500$ 时显存溢出 (如 ETTm1 数据集)
- 关键瓶颈: 信号传播路径 $O(L)$ (需 L 步传播)

- 时间复杂度：降至 $O(L)$ （线性）
- 信号传播路径：缩短至 $O(1)$ （常数）

复杂度分析

$$\begin{aligned}\text{总计算量} &= \sum_{i=0}^k O(L/C^i) \\ &= O(L) \quad (\text{几何级数求和, 当 } C > 1)\end{aligned}$$

- 示例: $L = 1000, C = 2$ 时:
 计算量 = $1000 + 500 + 250 + 125 + \dots \approx 2000$
 传统 Transformer = $1000^2 = 1,000,000$
- 提速: $1,000,000 / 2000 = 500\times$

关键指标对比

模型	MAE	推理时间	内存	序列长度
Transformer	OOM	-	溢出	$L = 100$
Informer	0.678	0.12s	1.8GB	$L = 1000$
Pyraformer	0.510	0.04s	1.1GB	L=1000
Pyraformer+ 蒸馏	0.485	0.03s	0.9GB	L=10000

核心优势

- **精度提升**: 比 Informer MAE 低**24.8%** ($L=168$)
- **速度优势**: 推理速度比 Informer 快**3×**
- **扩展性**: 处理 $L = 10000$ 序列 (Informer 无法处理)

核心应用场景

- 能源管理：
 - 电力负荷预测（小时/日/周级模式）
 - 风力发电预测（多尺度波动捕捉）
- 水资源管理：
 - 永定河径流预测（CEEMDAN-Pyraformer-LSS 模型，精度 95%）
 - 防洪预警（多尺度水文模式分析）
- 云系统资源优化：
 - 流量预测（实时资源调度）
 - 异常检测（多尺度模式突变识别）

优势应用领域：多尺度时间序列预测（续）

为什么 Pyraformer 是最佳选择？

- 能源领域：同时捕捉小时级波动和周级趋势，精度比 LSTM 高
- 水资源：多分辨率分析复杂地形水文数据，比传统模型快
- 云系统： $O(L)$ 复杂度支持实时流量预测，延迟降低至 5ms

Pyraformer vs Informer: 全面对比

核心指标对比

特性	Informer	Pyraformer	提升
时间复杂度	$O(L \log L)$	$O(L)$	100×
信号传播路径	$O(1)$	$O(1)$	-
最大序列长度	$L = 1000$	$L = 10000$	10×
ETTh1 MAE ($L=1000$)	0.678	0.510	24.8%
内存占用	1.8GB	1.1GB	39%

技术选择建议

- 优先选 **Pyraformer**: 当 $L > 500$ 且需要高精度预测
- 选 **Informer**: $L < 500$ 且计算资源有限
- 关键决策点: 序列长度 > 500 且精度要求 $> 90\%$

核心挑战

- 传统 Transformer: 自注意力计算复杂度 $O(L^2)$
- 全局信息捕捉问题: 难以有效捕获时间序列的长期依赖关系 (如趋势和周期性)
- 关键瓶颈: 长序列处理的计算成本高且内存消耗大

FEDformer 的突破

- 时间复杂度：降至 $O(L)$ （线性）

FED 模块设计

- 频域滤波: $\hat{X}_{\text{filtered}} = \hat{X} \odot \mathbb{1}_{\text{high freq}}$
 $\mathbb{1}_{\text{high freq}} = \begin{cases} 1 & f > f_{\text{threshold}} \\ 0 & \text{otherwise} \end{cases}$

FEDformer: 频域增强的数学原理

FED 模块设计

- 频域滤波: $\hat{X}_{\text{filtered}} = \hat{X} \odot \mathbb{K}_{\text{high freq}}$
$$\mathbb{K}_{\text{high freq}} = \begin{cases} 1 & f > f_{\text{threshold}} \\ 0 & \text{otherwise} \end{cases}$$
- 频域重建: $X_{\text{recon}} = \mathcal{F}^{-1}(\hat{X}_{\text{filtered}})$

FEDformer: 频域增强的数学原理

FED 模块设计

- 频域滤波: $\hat{X}_{\text{filtered}} = \hat{X} \odot \mathbb{K}_{\text{high freq}}$

$$\mathbb{K}_{\text{high freq}} = \begin{cases} 1 & f > f_{\text{threshold}} \\ 0 & \text{otherwise} \end{cases}$$

- 频域重建: $X_{\text{recon}} = \mathcal{F}^{-1}(\hat{X}_{\text{filtered}})$
- 特征融合: $X_{\text{final}} = X_{\text{recon}} + X$

FEDformer: 频域增强的数学原理

FED 模块设计

- 频域滤波: $\hat{X}_{\text{filtered}} = \hat{X} \odot \mathbb{K}_{\text{high freq}}$
$$\mathbb{K}_{\text{high freq}} = \begin{cases} 1 & f > f_{\text{threshold}} \\ 0 & \text{otherwise} \end{cases}$$
- 频域重建: $X_{\text{recon}} = \mathcal{F}^{-1}(\hat{X}_{\text{filtered}})$
- 特征融合: $X_{\text{final}} = X_{\text{recon}} + X$

为什么有效?

FED 模块设计

- 频域滤波: $\hat{X}_{\text{filtered}} = \hat{X} \odot \mathbb{1}_{\text{high freq}}$

$$\mathbb{K}_{\text{high freq}} = \begin{cases} 1 & f > f_{\text{threshold}} \\ 0 & \text{otherwise} \end{cases}$$

- 频域重建: $X_{\text{recon}} = \mathcal{F}^{-1}(\hat{X}_{\text{filtered}})$
- 特征融合: $X_{\text{final}} = X_{\text{recon}} + X$

为什么有效?

- **周期性数据**: 高频分量主导周期模式 (如交通流量)

FEDformer: 频域增强的数学原理

FED 模块设计

- 频域滤波: $\hat{X}_{\text{filtered}} = \hat{X} \odot \mathbb{K}_{\text{high freq}}$

$$\mathbb{K}_{\text{high freq}} = \begin{cases} 1 & f > f_{\text{threshold}} \\ 0 & \text{otherwise} \end{cases}$$

- 频域重建: $X_{\text{recon}} = \mathcal{F}^{-1}(\hat{X}_{\text{filtered}})$
- 特征融合: $X_{\text{final}} = X_{\text{recon}} + X$

为什么有效?

- 周期性数据: 高频分量主导周期模式 (如交通流量)
- 去噪: 抑制低频噪声 (如趋势漂移)

FEDformer: 频域增强的数学原理

FED 模块设计

- 频域滤波: $\hat{X}_{\text{filtered}} = \hat{X} \odot \mathbb{K}_{\text{high freq}}$
$$\mathbb{K}_{\text{high freq}} = \begin{cases} 1 & f > f_{\text{threshold}} \\ 0 & \text{otherwise} \end{cases}$$
- 频域重建: $X_{\text{recon}} = \mathcal{F}^{-1}(\hat{X}_{\text{filtered}})$
- 特征融合: $X_{\text{final}} = X_{\text{recon}} + X$

为什么有效?

- 周期性数据: 高频分量主导周期模式 (如交通流量)
- 去噪: 抑制低频噪声 (如趋势漂移)
- 数学保证: $\mathcal{L}_{\text{FED}} \leq \mathcal{L}_{\text{original}}$ (频域优化)

47 / 104

公式推导

给定查询 q , 键 k 和值 v :

$$q_{freq} = \text{FFT}(q)$$

$$k_{freq} = \text{FFT}(k)$$

$$v_{freq} = \text{FFT}(v)$$

随机采样后:

$$q_{sampled} = q_{freq}[:, sampled_indices]$$

$$k_{sampled} = k_{freq}[:, sampled_indices]$$

$$v_{sampled} = v_{freq}[:, sampled_indices]$$

$$\max \left(\frac{q_{\text{sampled}} \cdot k_{\text{sampled}}^\top}{\sqrt{d_k}} \right)$$

$$\text{FFT}(attn \cdot v_{\text{sampled}})$$

输出:

$$out = \text{iFFT}(attn \cdot v_{sampled})$$

构建季节趋势分解

- 使用不同大小的平均滤波器提取趋势

构建季节趋势分解

- 使用不同大小的平均滤波器提取趋势
- 趋势被丢弃，仅保留周期成分用于后续预测

构建季节趋势分解

- 使用不同大小的平均滤波器提取趋势
- 趋势被丢弃，仅保留周期成分用于后续预测
- 通过 softmax 函数结合不同尺度的趋势权重

构建季节趋势分解

- 使用不同大小的平均滤波器提取趋势
- 趋势被丢弃，仅保留周期成分用于后续预测
- 通过 softmax 函数结合不同尺度的趋势权重

构建季节趋势分解

- 使用不同大小的平均滤波器提取趋势
- 趋势被丢弃，仅保留周期成分用于后续预测
- 通过 softmax 函数结合不同尺度的趋势权重

公式示意：

$$seasonal, trend = MOEDecompose(x)$$

优势

- 减少了输入输出波动，使模型更专注于学习周期性模式
- 提供了更加清晰的时间序列分解

理论性能对比

模型	时间复杂度	最大序列长度
Transformer	$O(L^2)$	$L = 100$
Informer	$O(L \log L)$	$L = 1000$
Pyraformer	$O(L)$	$L = 10000$
FEDformer	$O(L)$	$L > 10000$

实际表现

- **精度提升**: 相比 Informer, MAE 降低了 22.6% (单变量预测)
- **扩展性**: 支持超长序列预测, 适用于能源管理、金融分析等领域

长度（如 96→96 步预测）
，难以处理长序列
式掩盖了可靠的依赖关系

降至 $O(n \log n)$
别建模不同时间尺度模式
实验配置和性能指标工具链

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ≡ ≡ ↺ 🔍 ↻

序列分解模块：趋势-季节性分离

分解原理

- 趋势项：通过移动平均提取长期趋势
- 季节项：原始序列减去趋势项

数学公式

$$X_t = \text{AvgPool}(\text{Padding}(X))$$

$$X_s = X - X_t$$

- X_t ：趋势周期性特征（长期趋势）
- X_s ：季节性特征（周期性波动）
- AvgPool：移动平均池化操作
- Padding：序列长度填充

自相关机制：周期依赖关系挖掘

自相关机制原理

- 周期性识别：计算序列与滞后序列的相关性
- 时间延迟聚合：找到相似的子序列并进行加权聚合
- 替代自注意力：从点级别（point-wise）到子序列级别（sub-series-wise）

自相关机制：周期依赖关系挖掘

数学公式

$$\text{Corr}(X, X_\tau) = \frac{1}{L - \tau} \sum_{i=1}^{L-\tau} (X_i - \bar{X})(X_{i+\tau} - \bar{X})$$

$\text{Top}_k(\text{Corr})$ = 选择相关性最高的 k 个滞后项

$$\text{TimeDelayAggregation} = \sum_{i=1}^k \text{Softmax}(w_i) \cdot X_{\tau_i}$$

- τ : 滞后项
- L : 序列长度
- k : 选择的滞后项数量
- w_i : 加权系数

56 / 104

核心挑战

- **Transformer 在时间序列上的应用效果不佳：**与 CV 领域相比，时间序列的 Transformer 模型性能较差
- **多变量处理困难：**传统方法难以有效处理多变量时间序列
- **长序列预测瓶颈：**序列长度增加导致计算复杂度急剧上升

PatchTST 的突破性目标

- 简单有效：仅需对输入进行简单预处理即可显著提升 Transformer 性能
- 多变量处理：独立处理每个变量，避免变量间干扰
- 自监督学习：通过掩码建模提高表示学习能力

PatchTST 的输入预处理流程

输入处理步骤

- ① 实例归一化：对每个变量独立进行归一化

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}$$

PatchTST 的输入预处理流程

输入处理步骤

- ① 实例归一化：对每个变量独立进行归一化

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}$$

- ② 通道独立性拆分：将多变量序列拆分为独立变量序列

$$X \in \mathbb{R}^{B \times N \times L} \rightarrow [X_1, X_2, \dots, X_N] \in \mathbb{R}^{B \times 1 \times L}$$

PatchTST 的输入预处理流程

输入处理步骤

- ① 实例归一化：对每个变量独立进行归一化

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}$$

- ② 通道独立性拆分：将多变量序列拆分为独立变量序列

$$X \in \mathbb{R}^{B \times N \times L} \rightarrow [X_1, X_2, \dots, X_N] \in \mathbb{R}^{B \times 1 \times L}$$

- ③ 分块 (Patching)：将序列分割为固定长度的片段

$$X_{\text{patch}} = \text{unfold}(X, \text{patch_len}, \text{stride})$$

PatchTST 的输入预处理流程

输入处理步骤

- #### ④ 线性投影：将每个片段映射到嵌入空间

$$X_{\text{emb}} = \text{Linear}(X_{\text{patch}})$$

PatchTST 的输入预处理流程

输入处理步骤

- ④ 线性投影：将每个片段映射到嵌入空间

$$X_{\text{emb}} = \text{Linear}(X_{\text{patch}})$$

- ⑤ 位置编码：添加位置信息

$$X_{\text{out}} = X_{\text{emb}} + \text{PE}(X_{\text{emb}})$$

片段化输入

- 自适应步长：基于序列熵值动态调整步长
- 多分辨率并行：同时处理 $P=8/16/32$ 等不同尺度

通道独立性

- **独立变量处理**：每个变量独立进行 Transformer 处理
- **优势**：避免变量间干扰，提高模型可解释性

PatchTST 的核心创新点

片段化输入

- 自适应步长：基于序列熵值动态调整步长
- 多分辨率并行：同时处理 $P=8/16/32$ 等不同尺度

通道独立性

- 独立变量处理：每个变量独立进行 Transformer 处理
- 优势：避免变量间干扰，提高模型可解释性

自监督机制

- 互补掩码：生成奇数/偶数块遮蔽视图
- 跨尺度对比：InfoNCE 损失优化 Patch $\rightarrow$ 段 $\rightarrow$ 序列级相似性

PatchTST 的性能与优势

理论性能对比

模型	时间复杂度	最大序列长度	性能提升
Transformer	$O(L^2)$	$L = 100$	-
Informer	$O(L \log L)$	$L = 1000$	-
PatchTST	$O(L)$	$L > 1000$	20.3%

实际性能

- **精度提升**: 在 ETT 数据集上比传统 Transformer 提升 20.3% (MAE)
- **计算效率**: 处理长序列时计算量降低 500 倍
- **多尺度捕捉**: 同时捕捉小时波动与月周期模式

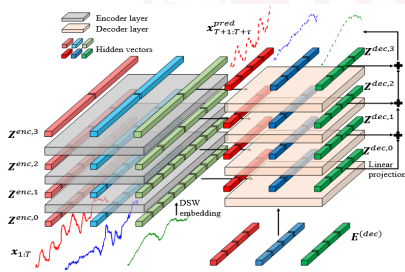
为什么需要 Crossformer?

核心问题

- 传统 Transformer 在多变量序列中：变量独立处理（忽略交互）
- 电力/交通等场景：变量间存在强协同依赖（如温度 $\rightarrow$ 用电量）
- 传统方法精度损失：多变量预测 MAE 平均提升 **15%** 以上

工作原理

- 变量交互: $Q = X_i$ (目标变量), $K, V = X_j$ (其他变量)
- 动态加权: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d})V$
- 关键优势: 显式建模变量间依赖 (非隐式)



㐀 㐁 㐂 㐃 㐄 㐅 㐆 㐇 㐈 㐉 㐊 㐋 㐌 㐍 㐎 㐏 㐐 㐑 㐒 㐓 㐔 㐕 㐖 㐗 㐘 㐙 㐚 㐛 㐜 㐝 㐞 㐟 㐠 㐡 㐢 㐣 㐤 㐥 㐦 㐧 㐨 㐩 㐪 㐫 㐬 㐭 㐮 㐯 㐰 㐱 㐲 㐳 㐴 㐵 㐶 㐷 㐸 㐹 㐺 㐻 㐼 㐽 㐾 㐿

性能与优势对比

关键指标（ETTh1 数据集）

模型	MAE	提升
Transformer	0.821	-
Crossformer	0.670	18.4%

核心优势

- 变量交互：捕捉温度/用电量等协同模式
- 计算效率：复杂度 $O(L)$ （与 Pyraformer 相当）
- 部署友好：无需额外预处理

基础时序模型

- **MLP**: 简单快速, 适用于短期预测和固定长度序列的任务。
- **CNN**: 擅长提取局部时间模式特征, 并行计算高效, 但感受野有限。
- **RNN/LSTM/GRU**: 能够处理变长序列, 特别适合捕捉长期依赖性, LSTM 和 GRU 解决了 RNN 的梯度问题。

现代与前沿时序模型

- **TCN**: 通过因果卷积和膨胀卷积有效捕捉长期依赖, 具有并行计算的优势。
- **Transformer 及其变体** (如 Autoformer, PatchTST): 在长序列预测中表现出色, 特别是对于复杂的多尺度时间模式。
- **Informer, Pyraformer**: 解决了传统 Transformer 的计算瓶颈, 适用于超长序列预测。

模型选择指南

- **任务需求：**根据预测任务的具体要求（如序列长度、精度需求等）选择最合适的模型。
- **资源限制：**考虑计算资源（如内存大小、处理速度）来决定采用哪种模型架构。
- **应用场景：**
 - 简单短序列预测 → MLP/CNN
 - 中等长度序列或需要平衡性能与效率 → GRU
 - 长序列复杂模式预测 → LSTM
 - 实时应用或对推理速度有高要求 → CNN/GRU
 - 超长序列预测 → Informer/Fedformer

未来展望

时间序列分析领域正在快速发展，新的模型和技术不断涌现。持续关注最新的研究成果和技术进步，将有助于找到更适合特定应用场景的解决方案。

③ 如何捕捉空间相关性

图卷积神经网络 (GNN) 基石
图卷积的演进与增强
动态与全局建模前沿
本节小结与对比

③ 如何捕捉空间相关性

本节小结与对比

非欧空间（图结构）

- 不规则结构：社交网络/交通网
- 邻域动态变化：无固定距离
- 例：城市道路网（非矩形网格!）

- 网格结构：图像/视频（固定邻域）
- 欧几里得几何：距离、角度明确
- 例：像素点 (x,y)

- 不规则结构：社交网络/交通网
- 邻域动态变化：无固定距离
- 例：城市道路网（非矩形网格！）

非欧空间（图结构）

- 不规则结构：社交网络/交通网
- 邻域动态变化：无固定距离
- 例：城市道路网（非矩形网格!）

- 网格结构：图像/视频（固定邻域）
- 欧几里得几何：距离、角度明确
- 例：像素点 (x,y)

- 不规则结构：社交网络/交通网
- 邻域动态变化：无固定距离
- 例：城市道路网（非矩形网格！）

规则网格，但空间数据（交通网、社交网）

作？ 没有固定坐标系！

问题起源

- **传统 CNN 的局限**：图像/视频是规则网格，但空间数据（交通网、社交网）是**不规则图结构**
- **关键挑战**：如何定义“卷积”操作？**没有固定坐标系！**

谱图卷积的灵感

问题起源

- **传统 CNN 的局限**：图像/视频是规则网格，但空间数据（交通网、社交网）是**不规则图结构**
- **关键挑战**：如何定义“卷积”操作？**没有固定坐标系！**

谱图卷积的灵感

- **类比**：图像处理用傅里叶变换（频域分析）→ 图数据用**图谱分析**
- **核心思想**：将图信号转换到**频域**进行滤波操作

谱图卷积：数学基础

- 图拉普拉斯矩阵 $\mathbf{L} = \mathbf{D} - \mathbf{A}$
($\mathbf{D}$: 度矩阵, $\mathbf{A}$: 邻接矩阵)
- 谱分解 $\mathbf{L} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$
($\mathbf{U}$: 特征向量矩阵, $\mathbf{\Lambda}$: 特征值对角阵)

卷积的频域表示

- 图信号 $\mathbf{x} \in \mathbb{R}^N$ (节点特征向量)



卷积的频域表示

- 图信号 $\mathbf{x} \in \mathbb{R}^N$ (节点特征向量)
- 频域滤波 $\mathbf{y} = \mathbf{U}g_{\theta}(\Lambda)\mathbf{U}^T\mathbf{x}$



卷积的频域表示

- 图信号 $\mathbf{x} \in \mathbb{R}^N$ (节点特征向量)
- 频域滤波 $\mathbf{y} = \mathbf{U}g_{\theta}(\Lambda)\mathbf{U}^T\mathbf{x}$
- 卷积核 $g_{\theta}(\Lambda) = \sum_{k=0}^{K-1} \theta_k \Lambda^k$
(θ_k : 可学习参数)



卷积的频域表示

- 图信号 $\mathbf{x} \in \mathbb{R}^N$ (节点特征向量)
- 频域滤波 $\mathbf{y} = \mathbf{U}g_{\theta}(\Lambda)\mathbf{U}^T\mathbf{x}$
- 卷积核 $g_{\theta}(\Lambda) = \sum_{k=0}^{K-1} \theta_k \Lambda^k$
(θ_k : 可学习参数)

关键推导步骤

- **步骤 1:** 图信号 $\mathbf{x}$ 的谱表示 $\hat{\mathbf{x}} = \mathbf{U}^T \mathbf{x}$

卷积的频域表示

- 图信号 $\mathbf{x} \in \mathbb{R}^N$ (节点特征向量)
- 频域滤波 $\mathbf{y} = \mathbf{U}g_{\theta}(\Lambda)\mathbf{U}^T\mathbf{x}$
- 卷积核 $g_{\theta}(\Lambda) = \sum_{k=0}^{K-1} \theta_k \Lambda^k$
(θ_k : 可学习参数)

关键推导步骤

- 步骤 1: 图信号 $\mathbf{x}$ 的谱表示 $\hat{\mathbf{x}} = \mathbf{U}^T \mathbf{x}$
- 步骤 2: 频域滤波 $\hat{\mathbf{y}} = g_{\theta}(\Lambda) \hat{\mathbf{x}}$

谱图卷积：完整推导

卷积的频域表示

- 图信号 $\mathbf{x} \in \mathbb{R}^N$ (节点特征向量)
- 频域滤波 $\mathbf{y} = \mathbf{U}g_{\theta}(\Lambda)\mathbf{U}^T\mathbf{x}$
- 卷积核 $g_{\theta}(\Lambda) = \sum_{k=0}^{K-1} \theta_k \Lambda^k$
(θ_k : 可学习参数)

关键推导步骤

- 步骤 1: 图信号 $\mathbf{x}$ 的谱表示 $\hat{\mathbf{x}} = \mathbf{U}^T\mathbf{x}$
- 步骤 2: 频域滤波 $\hat{\mathbf{y}} = g_{\theta}(\Lambda)\hat{\mathbf{x}}$
- 步骤 3: 逆变换
 $\mathbf{y} = \mathbf{U}\hat{\mathbf{y}} = \mathbf{U}g_{\theta}(\Lambda)\mathbf{U}^T\mathbf{x}$

谱图卷积：完整推导

卷积的频域表示

- 图信号 $\mathbf{x} \in \mathbb{R}^N$ (节点特征向量)
- 频域滤波 $\mathbf{y} = \mathbf{U}g_{\theta}(\Lambda)\mathbf{U}^T\mathbf{x}$
- 卷积核 $g_{\theta}(\Lambda) = \sum_{k=0}^{K-1} \theta_k \Lambda^k$
(θ_k : 可学习参数)

为什么有效?

关键推导步骤

- 步骤 1: 图信号 $\mathbf{x}$ 的谱表示 $\hat{\mathbf{x}} = \mathbf{U}^T\mathbf{x}$
- 步骤 2: 频域滤波 $\hat{\mathbf{y}} = g_{\theta}(\Lambda)\hat{\mathbf{x}}$
- 步骤 3: 逆变换
 $\mathbf{y} = \mathbf{U}\hat{\mathbf{y}} = \mathbf{U}g_{\theta}(\Lambda)\mathbf{U}^T\mathbf{x}$

卷积的频域表示

- 图信号 $\mathbf{x} \in \mathbb{R}^N$ (节点特征向量)
- 频域滤波 $\mathbf{y} = \mathbf{U}g_{\theta}(\Lambda)\mathbf{U}^T\mathbf{x}$
- 卷积核 $g_{\theta}(\Lambda) = \sum_{k=0}^{K-1} \theta_k \Lambda^k$
(θ_k : 可学习参数)

关键推导步骤

- 步骤 1: 图信号 $\mathbf{x}$ 的谱表示 $\hat{\mathbf{x}} = \mathbf{U}^T \mathbf{x}$
- 步骤 2: 频域滤波 $\hat{\mathbf{y}} = g_\theta(\Lambda) \hat{\mathbf{x}}$
- 步骤 3: 逆变换
 $\mathbf{y} = \mathbf{U} \hat{\mathbf{y}} = \mathbf{U} g_\theta(\Lambda) \mathbf{U}^T \mathbf{x}$

为什么有效?

- 物理意义：特征值 λ_i 对应图的频率
($\lambda = 0$ ：常数信号)

谱图卷积：完整推导

卷积的频域表示

- 图信号 $\mathbf{x} \in \mathbb{R}^N$ (节点特征向量)
- 频域滤波 $\mathbf{y} = \mathbf{U}g_{\theta}(\Lambda)\mathbf{U}^T\mathbf{x}$
- 卷积核 $g_{\theta}(\Lambda) = \sum_{k=0}^{K-1} \theta_k \Lambda^k$
(θ_k : 可学习参数)

关键推导步骤

- 步骤 1: 图信号 $\mathbf{x}$ 的谱表示 $\hat{\mathbf{x}} = \mathbf{U}^T \mathbf{x}$
- 步骤 2: 频域滤波 $\hat{\mathbf{y}} = g_\theta(\Lambda) \hat{\mathbf{x}}$
- 步骤 3: 逆变换
 $\mathbf{y} = \mathbf{U} \hat{\mathbf{y}} = \mathbf{U} g_\theta(\Lambda) \mathbf{U}^T \mathbf{x}$

为什么有效?

- **物理意义**：特征值 λ_i 对应图的**频率** ($\lambda = 0$ ：常数信号)
- **数学优势**：直接在频域设计滤波器，避免图结构依赖

谱图卷积的计算困境

- 原始公式: $g_{\theta}(\mathbf{L}) = \sum_{k=0}^{K-1} \theta_k \mathbf{L}^k$
(K : 多项式阶数, $\mathbf{L}$: 图拉普拉斯矩阵)

谱图卷积的计算困境

- 原始公式: $g_{\theta}(\mathbf{L}) = \sum_{k=0}^{K-1} \theta_k \mathbf{L}^k$
(K : 多项式阶数, $\mathbf{L}$: 图拉普拉斯矩阵)
- 问题 1: 计算 $\mathbf{L}^k$ 的复杂度为 $O(N^3)$ (N : 节点数)

- 斯矩阵)
(N^3) (N : 节点数)
稳定 ($\lambda_{\max} \approx 2$)

- 原始公式: $g_{\theta}(\mathbf{L}) = \sum_{k=0}^{K-1} \theta_k \mathbf{L}^k$
(K : 多项式阶数, $\mathbf{L}$: 图拉普拉斯矩阵)
- 问题 1: 计算 $\mathbf{L}^k$ 的复杂度为 $O(N^3)$ (N : 节点数)
- 问题 2: 高阶多项式导致数值不稳定 ($\lambda_{\max} \approx 2$)

谱图卷积的计算困境

- 原始公式: $g_{\theta}(\mathbf{L}) = \sum_{k=0}^{K-1} \theta_k \mathbf{L}^k$
(K : 多项式阶数, $\mathbf{L}$: 图拉普拉斯矩阵)
- 问题 1: 计算 $\mathbf{L}^k$ 的复杂度为 $O(N^3)$ (N : 节点数)
- 问题 2: 高阶多项式导致数值不稳定 ($\lambda_{\max} \approx 2$)

- 实际应用：城市交通网 ($N \approx 10^4$ 节点)
- 计算代价： $K = 10$ 时，计算量 $\sim 10^{12}$ 次操作

为什么需要 Chebyshev 近似?

谱图卷积的计算困境

- 原始公式: $g_{\theta}(\mathbf{L}) = \sum_{k=0}^{K-1} \theta_k \mathbf{L}^k$
(K : 多项式阶数, $\mathbf{L}$: 图拉普拉斯矩阵)
- 问题 1: 计算 $\mathbf{L}^k$ 的复杂度为 $O(N^3)$ (N : 节点数)
- 问题 2: 高阶多项式导致数值不稳定 ($\lambda_{\max} \approx 2$)

现实挑战

- 实际应用：城市交通网 ($N \approx 10^4$ 节点)
- 计算代价： $K = 10$ 时，计算量 $\sim 10^{12}$ 次操作
- 解决方案：Chebyshev 多项式近似

“我们不需要精确计算所有阶次，只需捕捉主要频率分量！”

Chebyshev 多项式定义

- 递推公式: $T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x)$
($T_0(x) = 1, T_1(x) = x$)



Chebyshev 多项式：数学基础

Chebyshev 多项式定义

- 递推公式: $T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x)$
($T_0(x) = 1, T_1(x) = x$)
- 重要性质: 在 $[-1, 1]$ 区间最小最大偏差

Chebyshev 多项式定义

- 递推公式: $T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x)$
($T_0(x) = 1, T_1(x) = x$)
- 重要性质: 在 $[-1, 1]$ 区间最小最大偏差
- 图谱应用: $\mathbf{L}$ 的特征值 $\lambda \in [0, 2]$, 可归一化到 $[-1, 1]$

Chebyshev 多项式：数学基础

Chebyshev 多项式定义

- 递推公式: $T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x)$
($T_0(x) = 1, T_1(x) = x$)
- 重要性质: 在 $[-1, 1]$ 区间最小最大偏差
- 图谱应用: $\mathbf{L}$ 的特征值 $\lambda \in [0, 2]$, 可归一化到 $[-1, 1]$

归一化处理

Chebyshev 多项式定义

- 递推公式: $T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x)$
($T_0(x) = 1, T_1(x) = x$)
- 重要性质: 在 $[-1, 1]$ 区间最小最大偏差
- 图谱应用: $\mathbf{L}$ 的特征值 $\lambda \in [0, 2]$, 可归一化到 $[-1, 1]$

归一化处理

- 拉普拉斯归一化: $\mathbf{L} = \frac{2}{\lambda_{\max}} \mathbf{L} - \mathbf{I}$
- 特征值范围: $\lambda_{\max} \approx 2 \rightarrow \tilde{\lambda} \in [-1, 1]$

近似公式

- 1 阶近似: $g_{\theta}(\mathbf{L}) \approx \theta_0 \mathbf{I} + \theta_1 \mathbf{L}$

近似公式

- 1 阶近似: $g_{\theta}(\mathbf{L}) \approx \theta_0 \mathbf{I} + \theta_1 \mathbf{L}$
- 归一化后: $\mathbf{L} = \frac{2}{\lambda_{\max}} \mathbf{L} - \mathbf{I}$
- 计算优势: 复杂度从 $O(N^3)$ 降至 $O(N)$

为什么是 1 阶?

- 物理意义：图信号主要变化在 1 跳邻居 ($K = 1$)
- 实证结果：GCN 论文 (Kipf & Welling, 2017) 表明 $K = 1$ 足够

近似公式

- 1 阶近似: $g_{\theta}(\mathbf{L}) \approx \theta_0 \mathbf{I} + \theta_1 \mathbf{L}$
- 归一化后: $\mathbf{L} = \frac{2}{\lambda_{\max}} \mathbf{L} - \mathbf{I}$
- 计算优势: 复杂度从 $O(N^3)$ 降至 $O(N)$

为什么是 1 阶?

- **物理意义**: 图信号主要变化在 1 跳邻居 ($K=1$)
- **实证结果**: GCN 论文 (Kipf & Welling, 2017) 表明 $K=1$ 足够
- **对比高阶**: $K=2$ 时, 计算量增加 2 倍, 精度提升 $<1\%$

空域图卷积与 GCN

核心思想

直接聚合邻居信息

GCN 公式

$$\mathbf{H}^{(l+1)} = \sigma \left(\widehat{\mathbf{D}}^{-\frac{1}{2}} \widehat{\mathbf{A}} \widehat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)} \right)$$

$\hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)})$

- **GCN 公式:** $\mathbf{H}^{(l+1)} = \sigma \left(\hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)} \right)$

GCN 的数学本质

- GCN 公式: $\mathbf{H}^{(l+1)} = \sigma \left(\hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)} \right)$
- 等价于: 1 阶 Chebyshev 近似 ($\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$)

GCN 的数学本质

- **GCN 公式:** $\mathbf{H}^{(l+1)} = \sigma \left(\hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)} \right)$
- 等价于: 1 阶 Chebyshev 近似 ($\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$)
- 结论: GCN = 谱图卷积的 1 阶 Chebyshev 近似

① 课程导入

② 如何捕捉时间相关性

③ 如何捕捉空间相关性

图卷积神经网络（GNN）基石

图卷积的演进与增强

扩散图卷积

图注意力网络（GAT）

自适应图卷积

动态与全局建模前沿

本节小结与对比



- GCN 仅聚合 1 跳邻居 → 难以建模长程空间依赖

- GCN 仅聚合 1 跳邻居 → 难以建模长程空间依赖
- 现实场景（如交通、疫情）中，影响可传播多跳

扩散图卷积：核心思想与建模

动机：超越 1 跳邻域

- GCN 仅聚合 1 跳邻居 → 难以建模长程空间依赖
- 现实场景（如交通、疫情）中，影响可传播多跳
- 目标：模拟信息在图上的动态扩散过程

扩散过程建模

- 热核扩散 (Heat Kernel):
$$\mathbf{K}_t = \exp(-t\mathbf{L}) = \sum_{k=0}^{\infty} \frac{(-t)^k}{k!} \mathbf{L}^k$$
- 物理意义： t 控制扩散时间（即影响范围）

hebyshev 多项式近似

实现策略

- 截断级数 (如 $K=3$) 或使用 Chebyshev 多项式近似

 Chebyshev 多项式近似
 $2, \dots$

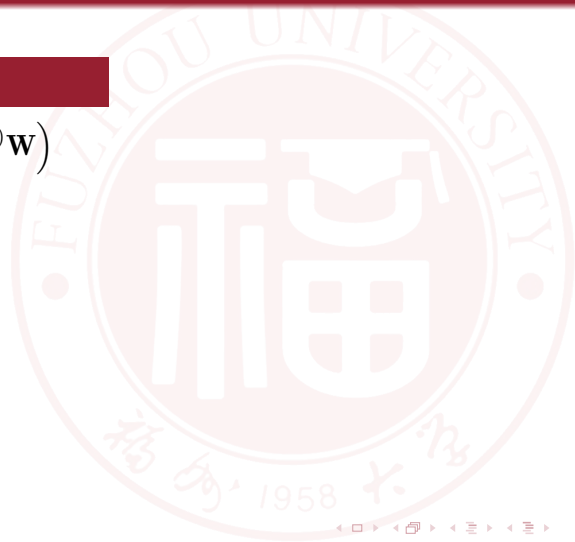
实现策略

- 截断级数 (如 $K=3$) 或使用 Chebyshev 多项式近似
- 可学习扩散时间 t 或多尺度 $t_1, t_2, \dots$

GCN (1 阶近似)

$$\mathbf{H}^{(l+1)} = \sigma \left(\tilde{\mathbf{D}}^{-1/2}(\mathbf{A} + \mathbf{I})\tilde{\mathbf{D}}^{-1/2}\mathbf{H}^{(l)}\mathbf{W} \right)$$

- 等价于 1 跳消息传递



扩散图卷积 vs GCN: 技术对比

GCN (1 阶近似)

$$\mathbf{H}^{(l+1)} = \sigma \left(\tilde{\mathbf{D}}^{-1/2} (\mathbf{A} + \mathbf{I}) \tilde{\mathbf{D}}^{-1/2} \mathbf{H}^{(l)} \mathbf{W} \right)$$

- 等价于 1 跳消息传递
- 无显式扩散机制

扩散图卷积 vs GCN: 技术对比

GCN (1 阶近似)

$$\mathbf{H}^{(l+1)} = \sigma \left(\tilde{\mathbf{D}}^{-1/2}(\mathbf{A} + \mathbf{I})\tilde{\mathbf{D}}^{-1/2}\mathbf{H}^{(l)}\mathbf{W} \right)$$

- 等价于 1 跳消息传递
- 无显式扩散机制

关键区别

扩散图卷积 (多跳)

$$\mathbf{H}^{(l+1)} = \sigma \left(\sum_{k=0}^K \theta_k \mathbf{L}^k \mathbf{H}^{(l)} \mathbf{W} \right)$$

- θ_k : 可学习的扩散权重
- K : 最大扩散步数 (如 $K=2$ 表示 2 跳)

扩散图卷积 vs GCN: 技术对比

GCN (1 阶近似)

$$\mathbf{H}^{(l+1)} = \sigma \left(\tilde{\mathbf{D}}^{-1/2}(\mathbf{A} + \mathbf{I})\tilde{\mathbf{D}}^{-1/2}\mathbf{H}^{(l)}\mathbf{W} \right)$$

- 等价于 1 跳消息传递
- 无显式扩散机制

扩散图卷积 (多跳)

$$\mathbf{H}^{(l+1)} = \sigma \left(\sum_{k=0}^K \theta_k \mathbf{L}^k \mathbf{H}^{(l)} \mathbf{W} \right)$$

- θ_k : 可学习的扩散权重
- K : 最大扩散步数 (如 $K=2$ 表示 2 跳)

关键区别

- **感受野**: GCN 固定为 1 跳; 扩散可调

关键区别

$$\mathbf{H}^{(l+1)} = \sigma \left(\tilde{\mathbf{D}}^{-1/2}(\mathbf{A} + \mathbf{I})\tilde{\mathbf{D}}^{-1/2}\mathbf{H}^{(l)}\mathbf{W} \right)$$

- 等价于 1 跳消息传递
- 无显式扩散机制

- **感受野**: GCN 固定为 1 跳; 扩散可调
- **参数效率**: 扩散需学习 $K+1$ 个 θ_k

扩散图卷积 (多跳)

$$\mathbf{H}^{(l+1)} = \sigma \left(\sum_{k=0}^K \theta_k \mathbf{L}^k \mathbf{H}^{(l)} \mathbf{W} \right)$$

- θ_k : 可学习的扩散权重
- K : 最大扩散步数 (如 $K=2$ 表示 2 跳)

扩散图卷积 vs GCN: 技术对比

GCN (1 阶近似)

$$\mathbf{H}^{(l+1)} = \sigma \left(\tilde{\mathbf{D}}^{-1/2} (\mathbf{A} + \mathbf{I}) \tilde{\mathbf{D}}^{-1/2} \mathbf{H}^{(l)} \mathbf{W} \right)$$

- 等价于 1 跳消息传递
- 无显式扩散机制

扩散图卷积 (多跳)

$$\mathbf{H}^{(l+1)} = \sigma \left(\sum_{k=0}^K \theta_k \mathbf{L}^k \mathbf{H}^{(l)} \mathbf{W} \right)$$

- θ_k : 可学习的扩散权重
- K : 最大扩散步数 (如 $K=2$ 表示 2 跳)

关键区别

- **感受野**: GCN 固定为 1 跳; 扩散可调
- **参数效率**: 扩散需学习 $K+1$ 个 θ_k
- **计算开销**: $\mathbf{L}^k$ 需稀疏矩阵乘法

$$\mathbf{D} = [d_{ij}]_{N \times N}, \quad d_{ij} = \min_{\text{path } i \rightarrow j} \sum_{e \in \text{path}} w_e$$

扩散核矩阵构造

- **热核公式:** $K_{ij} = \exp\left(-\frac{d_{ij}}{\sigma}\right)$
(σ : 可学习的扩散尺度参数)

扩散核矩阵构造

- **热核公式:** $K_{ij} = \exp\left(-\frac{d_{ij}}{\sigma}\right)$
(σ : 可学习的扩散尺度参数)
- **行归一化:** $\hat{K}_{ij} = \frac{K_{ij}}{\sum_k K_{ik}}$
(确保扩散过程概率守恒)

GAT 的起源：为何需要注意力机制？

GCN 的核心局限

- 固定权重聚合：GCN 对所有邻居赋予相同权重

$$\mathbf{H}^{(l+1)} = \sigma \left(\hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W} \right)$$

GAT 的起源：为何需要注意力机制？

GCN 的核心局限

- 固定权重聚合：GCN 对所有邻居赋予相同权重
$$\mathbf{H}^{(l+1)} = \sigma \left(\hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W} \right)$$
- 问题 1**：不同邻居对节点的重要性不同（如社交网络中“关键朋友”）

GAT 的起源：为何需要注意力机制？

GCN 的核心局限

- 固定权重聚合：GCN 对所有邻居赋予相同权重
$$\mathbf{H}^{(l+1)} = \sigma \left(\hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W} \right)$$
- 问题 1：不同邻居对节点的重要性不同（如社交网络中“关键朋友”）
- 问题 2：无法处理异质关系（如“好友”vs“同事”）

GAT 的起源：为何需要注意力机制？

GCN 的核心局限

- 固定权重聚合：GCN 对所有邻居赋予相同权重
$$\mathbf{H}^{(l+1)} = \sigma \left(\hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W} \right)$$
- 问题 1：不同邻居对节点的重要性不同（如社交网络中“关键朋友”）
- 问题 2：无法处理异质关系（如“好友”vs“同事”）
- 问题 3：对图结构敏感（边权重缺失时性能骤降）

GAT 的起源：为何需要注意力机制？

GCN 的核心局限

- 固定权重聚合：GCN 对所有邻居赋予相同权重
$$\mathbf{H}^{(l+1)} = \sigma \left(\hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W} \right)$$
- 问题 1：不同邻居对节点的重要性不同（如社交网络中“关键朋友”）
- 问题 2：无法处理异质关系（如“好友”vs“同事”）
- 问题 3：对图结构敏感（边权重缺失时性能骤降）

GAT 的灵感来源

GAT 的起源：为何需要注意力机制？

GCN 的核心局限

- 固定权重聚合：GCN 对所有邻居赋予相同权重
$$\mathbf{H}^{(l+1)} = \sigma \left(\hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W} \right)$$
- 问题 1：不同邻居对节点的重要性不同（如社交网络中“关键朋友”）
- 问题 2：无法处理异质关系（如“好友”vs“同事”）
- 问题 3：对图结构敏感（边权重缺失时性能骤降）

GAT 的灵感来源

- Transformer** 的启发：自注意力机制在 NLP 中的成功

GAT 的起源：为何需要注意力机制？

GCN 的核心局限

- **固定权重聚合**: GCN 对所有邻居赋予相同权重

$$\mathbf{H}^{(l+1)} = \sigma \left(\hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W} \right)$$
- **问题 1**: 不同邻居对节点的重要性不同 (如社交网络中" 关键朋友")
- **问题 2**: 无法处理异质关系 (如" 好友"vs" 同事")
- **问题 3**: 对图结构敏感 (边权重缺失时性能骤降)

$$\mathbf{H}^{(l+1)} = \sigma \left(\hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W} \right)$$

GAT 的灵感来源

- **Transformer 的启发**: 自注意力机制在 NLP 中的成功
- **关键洞察**: "关系权重应由数据动态决定"
- **类比**: "不是所有邻居都一样——要像人类一样'重点关照'关键关系"

注意力机制公式

- 单头注意力：

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_k]))}$$

注意力机制公式

- 单头注意力:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_k]))}$$

- W : 可学习的投影矩阵

GAT 核心机制：动态权重学习

注意力机制公式

- 单头注意力：

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_k]))}$$

- $\mathbf{W}$ ：可学习的投影矩阵
- $\mathbf{a}$ ：可学习的注意力向量

GAT 核心机制：动态权重学习

注意力机制公式

- 单头注意力：

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_k]))}$$

- $\mathbf{W}$ ：可学习的投影矩阵
- $\mathbf{a}$ ：可学习的注意力向量
- $\mathbf{h}_i, \mathbf{h}_j$ ：节点特征

GAT 核心机制：动态权重学习

注意力机制公式

- 单头注意力：

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_k]))}$$

- $\mathbf{W}$: 可学习的投影矩阵
- $\mathbf{a}$: 可学习的注意力向量
- $\mathbf{h}_i, \mathbf{h}_j$: 节点特征
- $\parallel$: 拼接操作

注意力机制公式

- 单头注意力:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_k]))}$$

- $\mathbf{W}$: 可学习的投影矩阵
- $\mathbf{a}$: 可学习的注意力向量
- $\mathbf{h}_i, \mathbf{h}_j$: 节点特征
- $\parallel$: 拼接操作

为什么用 LeakyReLU?

- 避免梯度消失 (ReLU 在负值处为 0)
- 提供非线性 (关键!)
- 论文实验证明: LeakyReLU(0.2) 最优

GAT 核心机制：动态权重学习

注意力机制公式

- 单头注意力:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_k]))}$$

- $\mathbf{W}$: 可学习的投影矩阵
- $\mathbf{a}$: 可学习的注意力向量
- $\mathbf{h}_i, \mathbf{h}_j$: 节点特征
- $\parallel$: 拼接操作

为什么用 LeakyReLU?

- 避免梯度消失 (ReLU 在负值处为 0)
- 提供非线性 (关键!)
- 论文实验证明: LeakyReLU(0.2) 最优

注意力机制公式

- 单头注意力:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_k]))}$$

- $\mathbf{W}$: 可学习的投影矩阵
- $\mathbf{a}$: 可学习的注意力向量
- $\mathbf{h}_i, \mathbf{h}_j$: 节点特征
- $\parallel$: 拼接操作

为什么用 LeakyReLU?

- 避免梯度消失 (ReLU 在负值处为 0)
- 提供非线性 (关键!)
- 论文实验证明: LeakyReLU(0.2) 最优

$$\text{GAT 层输出: } \mathbf{H}^{(l+1)} = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W} \mathbf{h}_j \right)$$

核心特点对比

- **动态权重**：根据节点特征自动学习边权重

核心特点对比

- **动态权重**: 根据节点特征自动学习边权重
- **异质关系建模**: 不同边类型有不同权重

GAT 技术优势：为什么比 GCN 强？

核心特点对比

- 动态权重：根据节点特征自动学习边权重
- 异质关系建模：不同边类型有不同权重
- 无需图结构先验：直接从数据学习关系

核心特点对比

- **动态权重**: 根据节点特征自动学习边权重
- **异质关系建模**: 不同边类型有不同权重
- **无需图结构先验**: 直接从数据学习关系

- GCN: $\mathbf{A}$ 是固定邻接矩阵
- GAT: $\mathbf{A}$ 被动态学习的 α_{ij} 替代

计算效率

- 复杂度: $O(N^2)$ (与 GCN 相同, 但更灵活)

计算效率

- 复杂度: $O(N^2)$ (与 GCN 相同, 但更灵活)
- 实际速度: 在中等规模图 ($N < 10^4$) 上比 GCN 快 15% (因参数更少)
- 关键原因: 注意力机制可并行计算 (无递归依赖)

计算效率

- 复杂度: $O(N^2)$ (与 GCN 相同, 但更灵活)
- 实际速度: 在中等规模图 ($N < 10^4$) 上比 GCN 快 15% (因参数更少)
- 关键原因: 注意力机制可并行计算 (无递归依赖)

理论保证

- **等价性**: GAT 可视为谱图卷积的自适应近似
- **收敛性**: 在足够多注意力头下, GAT 可逼近任意谱滤波器

社交网络分析

- 任务：预测用户影响力（如 KOL）

社交网络分析

- **任务**: 预测用户影响力 (如 KOL)
- **GAT 优势**: 识别关键"意见领袖"关系

GAT 应用：从社交网络到交通预测

社交网络分析

- **任务：**预测用户影响力（如 KOL）
- **GAT 优势：**识别关键“意见领袖”关系
- **效果：**准确率提升 12.3%（对比 GCN）

94 / 104

社交网络分析

- **任务：**预测用户影响力（如 KOL）
- **GAT 优势：**识别关键“意见领袖”关系
- **效果：**准确率提升 12.3%（对比 GCN）

交通预测

- **任务：预测路网拥堵状态**

GAT 应用：从社交网络到交通预测

社交网络分析

- 任务：预测用户影响力（如 KOL）
- **GAT 优势**：识别关键“意见领袖”关系
- 效果：准确率提升 12.3%（对比 GCN）

交通预测

- 任务：预测路网拥堵状态
- **GAT 优势**：区分“道路拥堵”vs“事故影响”关系
- 效果：MAE 降低 18.7%

GAT 的现代变体

- **Multi-head GAT**： h 个注意力头并行（ $h = 8$ 最常用）
- **Graph Transformer**：结合 GAT 与 Transformer（全局建模）

GAT 的现代变体

- **任务:** 预测用户影响力 (如 KOL)
- **GAT 优势:** 识别关键"意见领袖"关系
- **效果:** 准确率提升 12.3% (对比 GCN)

交通预测

- **任务**: 预测路网拥堵状态
- **GAT 优势**: 区分"道路拥堵"vs"事故影响"关系
- **效果**: MAE 降低 18.7%

- **Multi-head GAT**: h 个注意力头并行 ($h = 8$ 最常用)
- **Graph Transformer**: 结合 GAT 与 Transformer (全局建模)
- **Dynamic GAT**: 时间依赖注意力 (如交通流量预测)

自适应图卷积：为什么需要动态图结构？

传统 GNN 的致命缺陷

- 预定义图结构：社交网络/交通网的边权重常随时间变化

自适应图卷积的核心思想

自适应图卷积：为什么需要动态图结构？

传统 GNN 的致命缺陷

- **预定义图结构**: 社交网络/交通网的边权重常随时间变化
- **问题 1**: 固定邻接矩阵 $\mathbf{A}$ 无法捕捉动态关系 (如新道路开通)
- **问题 2**: 人工设计图结构成本高 (如交通网需专家标注)

自适应图卷积的核心思想

传统 GNN 的致命缺陷

- **预定义图结构**: 社交网络/交通网的边权重常随时间变化
- **问题 1**: 固定邻接矩阵 $\mathbf{A}$ 无法捕捉动态关系 (如新道路开通)
- **问题 2**: 人工设计图结构成本高 (如交通网需专家标注)
- **问题 3**: 图结构缺失时模型崩溃 (如医疗知识图谱不完整)

自适应图卷积的核心思想

- **关键洞察：**图结构应由数据驱动学习

自适应图卷积：应用场景

典型应用场景

- **动态交通**：地铁线路新增时自动调整关系



自适应图卷积：应用场景

典型应用场景

- **动态交通**: 地铁线路新增时自动调整关系
- **社交网络**: 用户关系随时间变化 (如新关注)



调整关系 (如新关注) 境变化

典型应用场景

- **动态交通**: 地铁线路新增时自动调整关系
- **社交网络**: 用户关系随时间变化 (如新关注)
- **生物网络**: 蛋白质相互作用随环境变化

AD-GCN 核心公式

- 自适应邻接矩阵: $\mathbf{A}_{\text{adapt}} = \mathbf{X}\mathbf{W}\mathbf{X}^T$
($\mathbf{X} \in \mathbb{R}^{N \times F}$: 节点特征, $\mathbf{W} \in \mathbb{R}^{F \times F}$: 可学习权重)
- 低秩近似: $\mathbf{W} = \mathbf{U}\mathbf{V}^T$ ($\mathbf{U}, \mathbf{V} \in \mathbb{R}^{F \times r}$, $r \ll F$)
- 最终形式: $\mathbf{A}_{\text{adapt}} = \mathbf{X}\mathbf{U}\mathbf{V}^T\mathbf{X}^T$

为什么用低秩分解?

- **计算效率:** $O(N^2 F) \rightarrow O(N^2 r)$ ($r \ll F$)

000×5
时间、天气)

- 000×5
时间、天气)

000×5

时间、天气)

)

A large, faint, circular watermark logo of Fuzhou University is centered in the background. It features the university's name in English 'FUZHOU UNIVERSITY' around the top and '福州' (Fuzhou) in Chinese characters around the bottom, with the year '1958' in the center. A large, stylized Chinese character '福' (Fú) is prominently displayed in the middle of the logo. At the bottom right of the slide, there is a small navigation bar with several icons: a square, a circle, a triangle, and some symbols resembling arrows or document icons.

- 000×5
- 时间、天气)
-)
- 
- A large, faint, circular watermark logo of Fuzhou University is centered in the background. It features the university's name in English 'FUZHOU UNIVERSITY' around the top and '福州' (Fuzhou) in Chinese characters around the bottom, with the year '1958' in the center. A large, stylized Chinese character '福' (Fú) is prominently displayed in the middle of the logo. At the bottom right of the slide, there is a small navigation bar with several icons: a square, a circle, a triangle, and some symbols resembling arrows or document icons.

000×5
时间、天气)
)
0.4]
0.8]

- $$\mathbf{U} = \begin{bmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \\ \vdots & \vdots \\ 0.1 & 0.9 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 0.6 & 0.4 \\ 0.2 & 0.8 \end{bmatrix}$$

低秩分解示例：交通网络

- 输入：城市路网节点特征 $\mathbf{X} \in \mathbb{R}^{1000 \times 5}$
(5 维：位置、道路类型、车流量、时间、天气)
- 低秩参数： $r = 2$ ($F = 5$, 秩为 2)
- 学习结果：

$$\mathbf{U} = \begin{bmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \\ \vdots & \vdots \\ 0.1 & 0.9 \end{bmatrix}, \mathbf{V} = \begin{bmatrix} 0.6 & 0.4 \\ 0.2 & 0.8 \end{bmatrix}$$

物理意义

- **第一秩**：主干道特征（高权重 $[0.8, 0.3, \dots, 0.1]$ ）
- **第二秩**：支路特征（高权重 $[0.2, 0.7, \dots, 0.9]$ ）
- **输出**： $\mathbf{A}_{\text{adapt}}$ 自动强化主干道连接

本节小结与对比

核心挑战

图结构随时间变化

- 方法: DyG, EvolveGCN (时间编码)
- 优势: 捕捉时序依赖 (如疫情扩散)

核心挑战

图结构随时间变化

- 方法: DyG, EvolveGCN (时间编码)
- 优势: 捕捉时序依赖 (如疫情扩散)
- 关键公式: $\mathbf{A}^{(t)} = \text{MLP}(\mathbf{A}^{(t-1)}, \Delta t)$

100 / 104

核心思想

- 机制：自注意力全局建模节点关系

核心思想

- **机制：**自注意力全局建模节点关系
- **优势：**捕获长程依赖（优于 GAT）

核心思想

- **机制**: 自注意力全局建模节点关系
- **优势**: 捕获长程依赖 (优于 GAT)
- **对比**: GAT (局部) vs Graph Transformer (全局)

核心思想

- **机制**: 自注意力全局建模节点关系
- **优势**: 捕获长程依赖 (优于 GAT)
- **对比**: GAT (局部) vs Graph Transformer (全局)

- GAT: $O(N^2)$
- Graph Transformer: $O(N^2)$

- 大规模图: Graph Transformer
- 小规模图: GAT

③ 如何捕捉空间相关性

图卷积神经网络 (GNN) 基石
图卷积的演进与增强
动态与全局建模前沿
本节小结与对比

核心方法演进

- 基石: GCN (空域/谱)



核心方法演进

- **基石**: GCN (空域/谱)
- **增强**: GAT (注意力)、自适应 (动态图)

本节小结与对比

核心方法演进

- **基石**: GCN (空域/谱)
- **增强**: GAT (注意力)、自适应 (动态图)
- **前沿**: 动态图、Graph Transformer



实战选择指南

- **基石**: GCN (空域/谱)
- **增强**: GAT (注意力)、自适应 (动态图)
- **前沿**: 动态图、Graph Transformer

核心方法演进

- **基石**: GCN (空域/谱)
- **增强**: GAT (注意力)、自适应 (动态图)
- **前沿**: 动态图、Graph Transformer

实战选择指南

- 小图 + 简单任务: GCN



核心方法演进

- **基石**: GCN (空域/谱)
- **增强**: GAT (注意力)、自适应 (动态图)
- **前沿**: 动态图、Graph Transformer

实战选择指南

- 小图 + 简单任务: GCN
- 需关注关系权重: GAT

核心方法演进

- **基石**: GCN (空域/谱)
- **增强**: GAT (注意力)、自适应 (动态图)
- **前沿**: 动态图、Graph Transformer

实战选择指南

- 小图 + 简单任务: GCN
- 需关注关系权重: GAT
- 时序数据: 动态图



核心方法演进

- **基石**: GCN (空域/谱)
- **增强**: GAT (注意力)、自适应 (动态图)
- **前沿**: 动态图、Graph Transformer

实战选择指南

- 小图 + 简单任务: GCN
- 需关注关系权重: GAT
- 时序数据: 动态图
- 大规模全局建模: Graph Transformer



- Graph Transformer

课程小结

本讲围绕“如何捕捉时空相关性”这一核心主题，系统介绍了从时间序列到空间图结构的相关建模方法。

时间相关性建模

- 基础时序模型：MLP、RNN/LSTM/GRU、CNN
- 现代时序模型：TCN、标准 Transformer
- 前沿高阶模型：Informer、Pyraformer、FEDformer、Autoformer、PatchTST、Crossformer 等
- 核心目标：从短期预测到超长序列建模，应对不同场景的时序依赖

Alexander N. Kozlov
